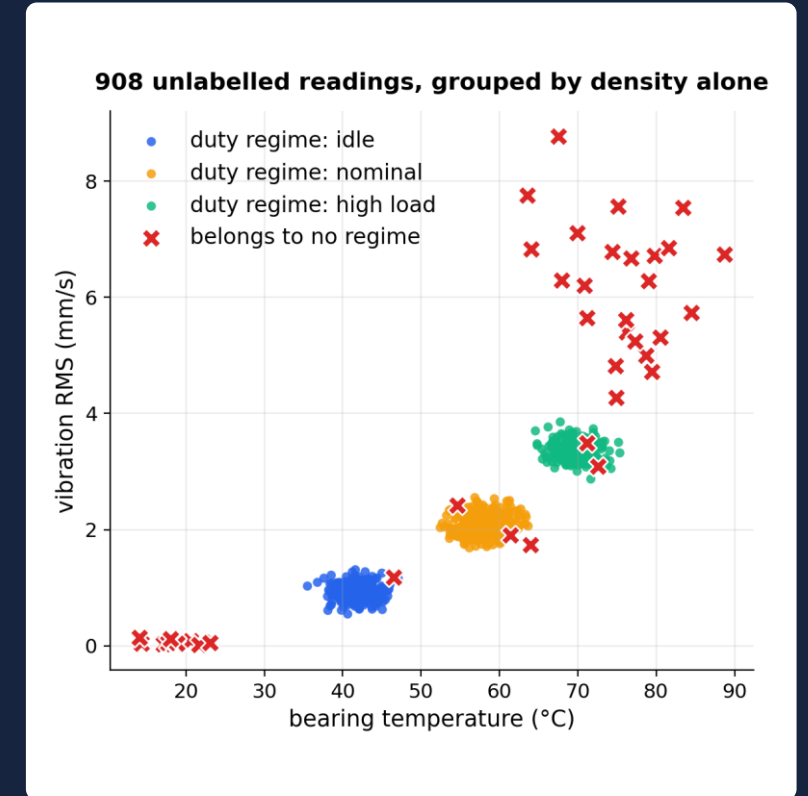


DBSCAN

Density-based clustering for machine condition monitoring

Finding operating regimes — and the readings that belong to none of them — in unlabelled sensor data.



No k to choose

density decides how many clusters exist

Any cluster shape

no assumption of round, equal blobs

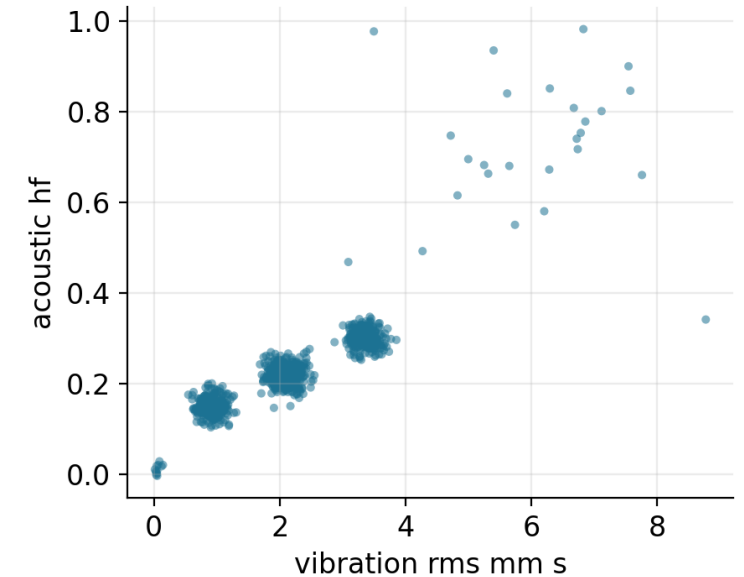
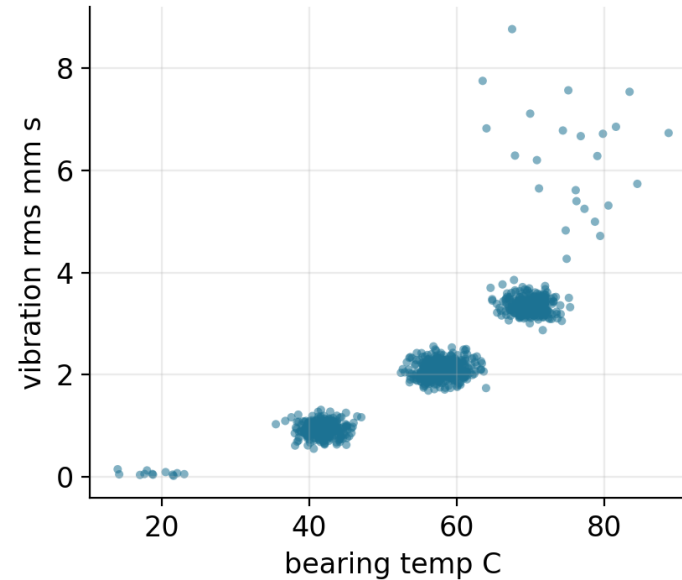
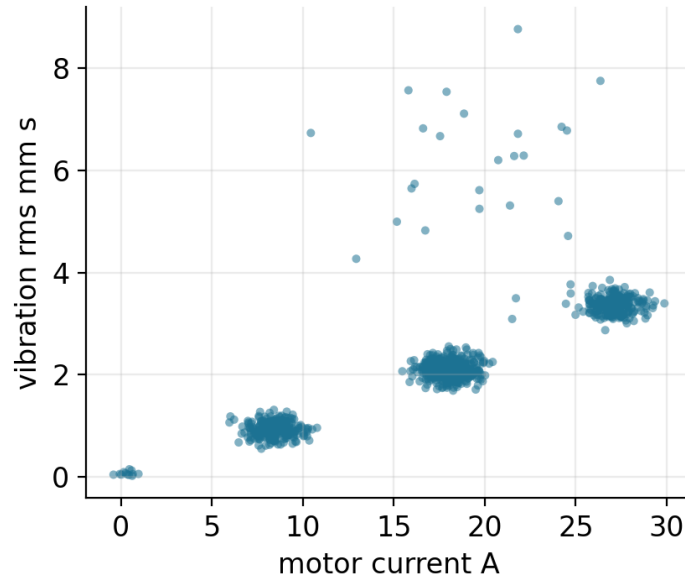
Noise is an output

clustering and anomaly detection at once

1

The problem: unlabelled machine data

A fleet of motor-driven process pumps · four channels · one reading per hour



What we have

908 hourly readings of vibration RMS, bearing temperature, motor current and high-frequency acoustic energy. No fault labels — failures are rare and the work orders are free text.

What we want

The duty regimes the fleet actually runs in, so alarm limits can be set per regime — and the readings that fit no regime at all.

Why not supervised?

There is nothing to train on. Labelling would need a fault history nobody recorded. So the structure has to come from the data itself.

2

Why not K-Means?

Three cases where the centroid assumption fails — same data, two algorithms

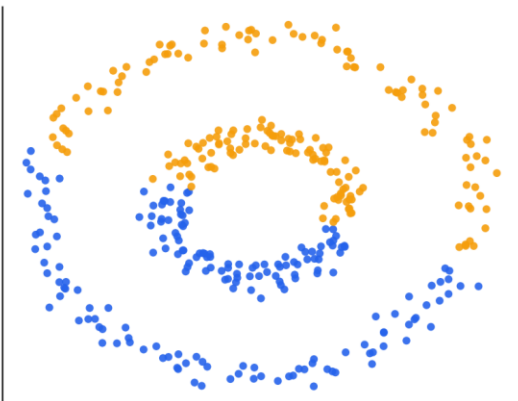
Two crescents

K-Means



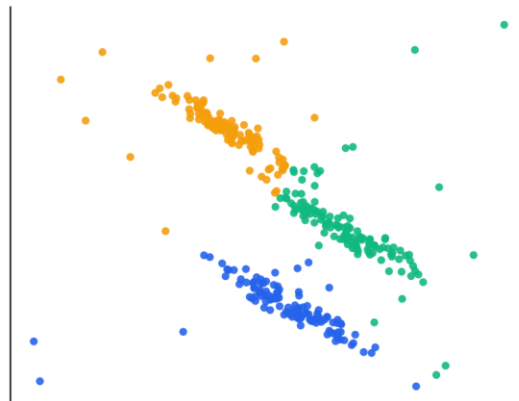
Concentric rings

K-Means



Elongated + scattered noise

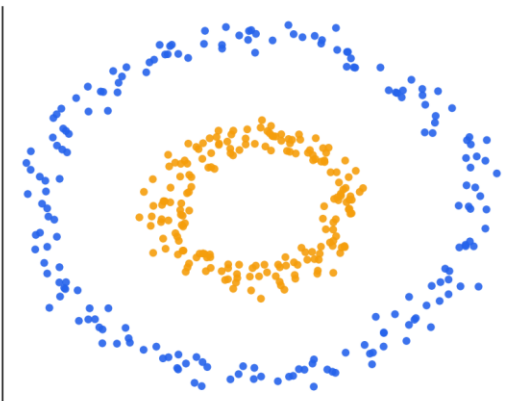
K-Means



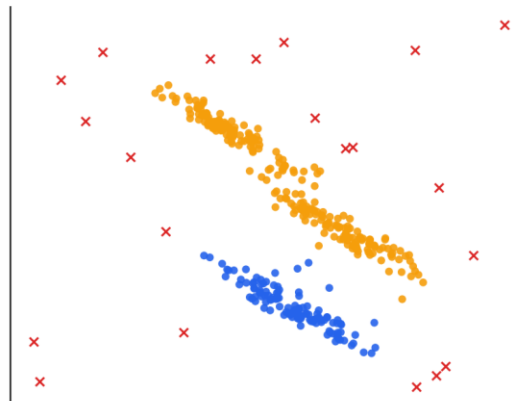
DBSCAN



DBSCAN



DBSCAN



K-Means must

- be told k in advance
- draw convex, straight-edged cells
- assign every point to some cluster

DBSCAN instead

- discovers the cluster count
- follows density, so any shape works
- may answer “none of them” — label -1

3

Two parameters, three kinds of point

Everything DBSCAN does follows from these definitions

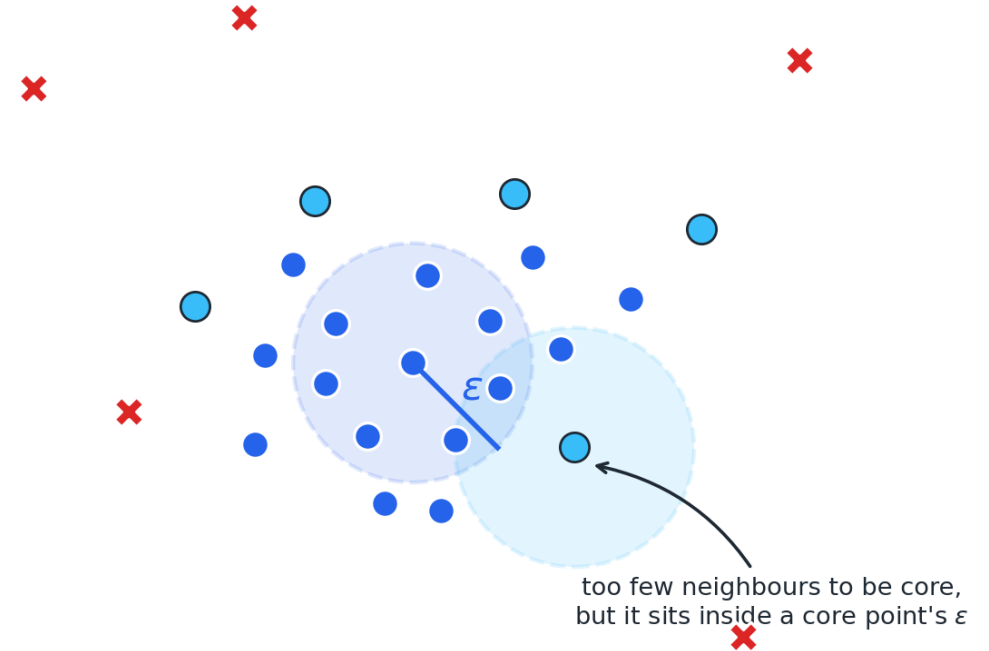
ϵ (eps)

the neighbourhood radius —
how close is “near”?

MinPts (min_samples)

how many neighbours make a region
dense

The three kinds of point ($\epsilon = 0.85$, MinPts = 4)



Core point

at least MinPts points lie within ϵ → sklearn: core_sample_indices_

Border point

too few neighbours of its own, but inside some core point's ϵ

Noise point

neither → sklearn: labels_ == -1. The anomaly.

- Core point — at least MinPts neighbours within ϵ
- Border point — too few neighbours, but inside a core's ϵ
- ✗ Noise — neither core nor reachable from one

4

How a cluster grows

Density-reachability: the rule is entirely local, which is why any shape works

1

Pick any unvisited point

count how many points lie within ϵ

2

Dense enough?

if $\geq \text{MinPts}$, it is a core point — start a new cluster

3

Expand

absorb its ϵ -neighbours; recurse on any that are themselves core

4

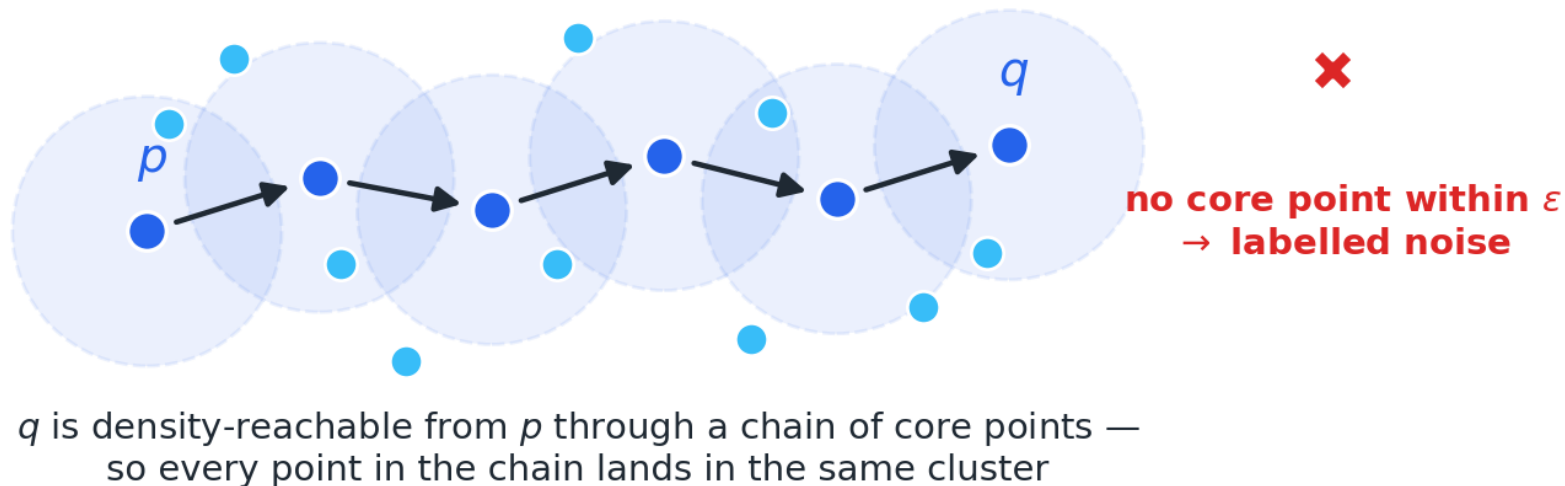
Exhausted?

close the cluster, move to the next unvisited point

5

Left over

points reached by nothing are labelled noise (-1)



Complexity

Each point is visited once and its ϵ -neighbourhood queried once, so DBSCAN costs about $O(n \log n)$ with a spatial index (kd-tree / ball-tree, chosen automatically by scikit-learn) and degrades toward $O(n^2)$ in high dimensions, where the index stops helping. Beyond roughly 10–15 features, reduce dimension first.

5

Choosing ϵ and MinPts

Do not guess ϵ — read it off the sorted k-distance curve

Step 1 — fix MinPts

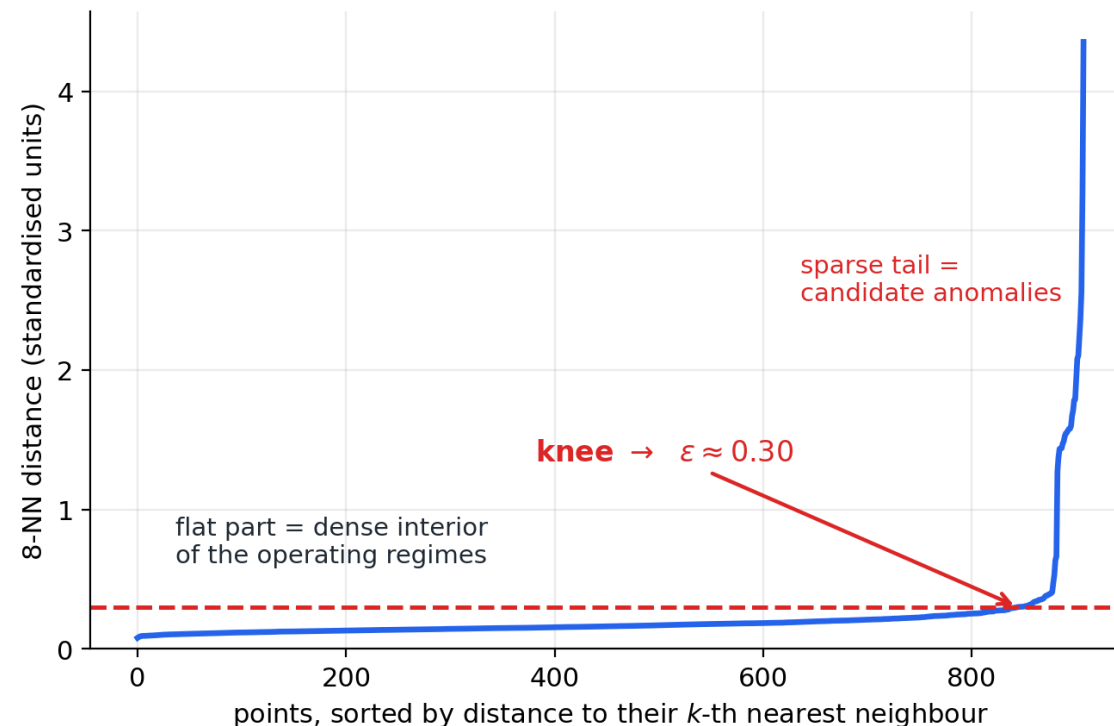
Rule of thumb: $\text{MinPts} \geq d + 1$, preferably $2 \cdot d$. With $d = 4$ channels we take $\text{MinPts} = 8$. Larger values mean more noise but more robust clusters — and MinPts is the less sensitive of the two knobs, so settle it first.

Step 2 — read ϵ off the knee

For every point, measure the distance to its k-th nearest neighbour. Sort ascending and plot. Points inside a dense regime sit on the flat part; outliers are in the tail. The knee is where “dense” stops meaning anything — here $\epsilon \approx 0.30$.

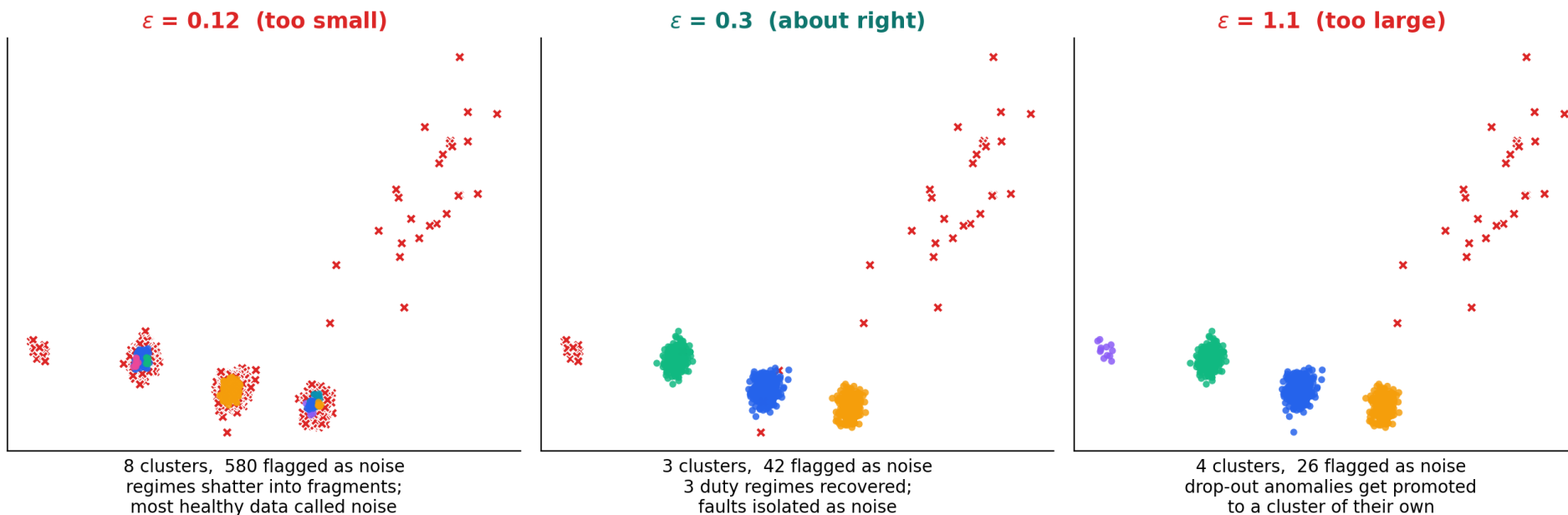
Step 0 — and never skip it: standardise

ϵ is a distance, so it is meaningless until the channels share a scale. On raw units — °C against an index in $[0,1]$ — the same $\epsilon = 0.30$ calls 768 of 908 readings noise. After `StandardScaler` it finds 3 clean regimes and 42 anomalies.



How much does ϵ matter? A great deal

Never report a DBSCAN result from a single ϵ — report the range over which the answer is stable



ϵ too small

Regimes shatter into fragments and most healthy data is called noise. Recall looks perfect — because it flags nearly everything.

The plateau

$\epsilon \approx 0.25$ – 0.30 : three clusters, ~5 % noise, silhouette 0.81. The answer not changing across a range of ϵ is the real evidence the structure is genuine.

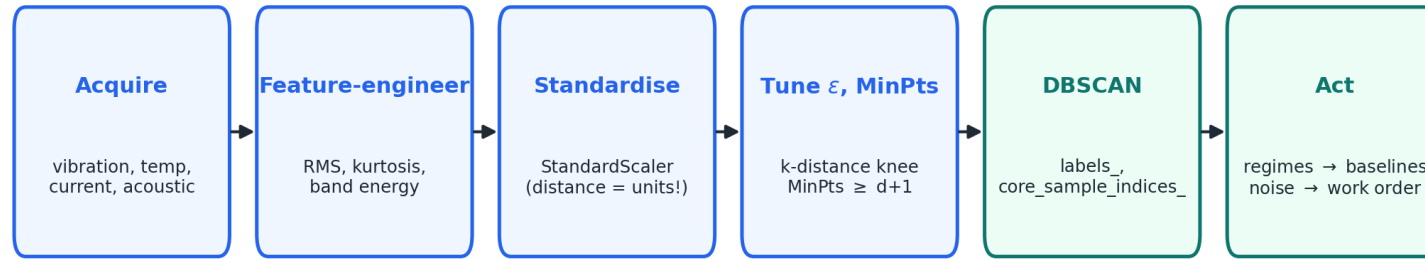
ϵ too large

At $\epsilon \geq 0.40$ the twelve sensor drop-outs are dense enough among themselves to become their own cluster — so they stop being flagged, and recall falls to 0.68.

7

Putting it to work with scikit-learn

From raw sensor archive to a maintenance work order



```

from sklearn.preprocessing import StandardScaler
from sklearn.cluster         import DBSCAN

X = StandardScaler().fit_transform(raw)

db = DBSCAN(eps=0.30, min_samples=8).fit(X)

labels      = db.labels_          # -1 = noise
core_idx    = db.core_sample_indices_
n_regimes   = len(set(labels) - {-1}) # → 3
alarms      = raw[labels == -1]      # → work orders
  
```

No .predict() — and that is not an oversight

DBSCAN is transductive: it labels the data it was fitted on. There is no centroid to compare a new reading to.

The production pattern

A new reading joins cluster c if it falls within ϵ of a core point of c ; otherwise it is -1 . That reproduces DBSCAN's own border-point rule in a few lines — see §8 of the notebook.

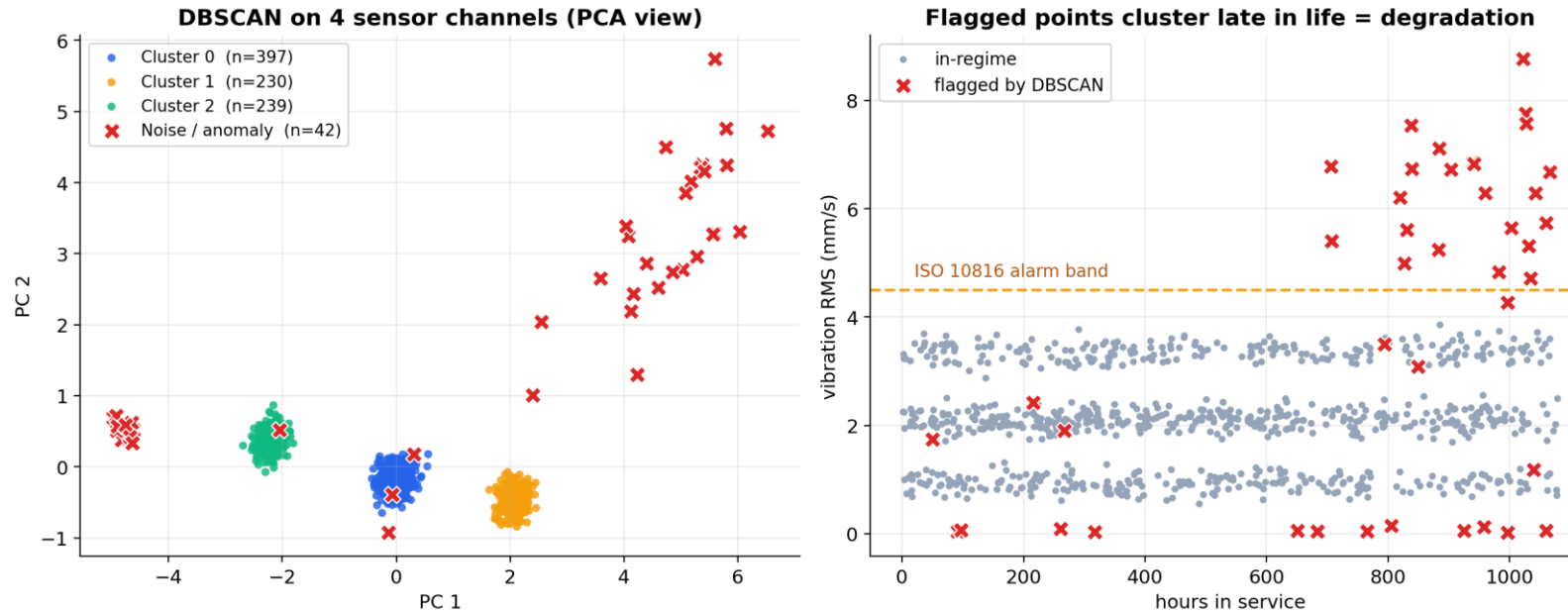
Persist the scaler with the model

Re-fitting StandardScaler on new data silently shifts the geometry, and every ϵ you tuned becomes wrong.

8

Result: three duty regimes and a work-order queue

DBSCAN on the four standardised channels, $\epsilon = 0.30$, MinPts = 8 — no labels used



3

duty regimes found automatically

1.00

recall — every injected fault was flagged

0.90

precision — 38 of 42 alarms were real

0.81

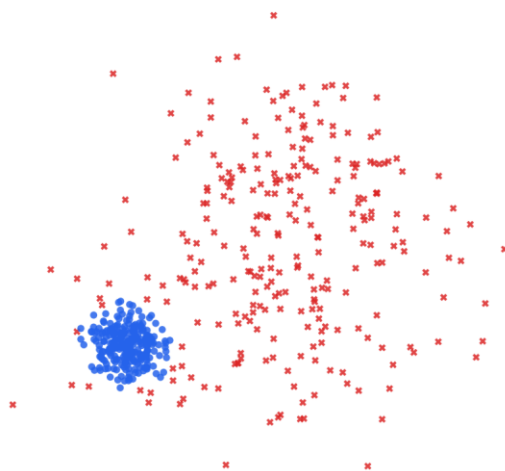
silhouette on the non-noise points

Adjusted Rand Index against the true duty regimes: 1.000 — the three clusters are exactly the three regimes. And 79 % of the flagged readings occur after 700 hours in service: the anomalies arrive in time order, which is what distinguishes a developing bearing fault from a random glitch.

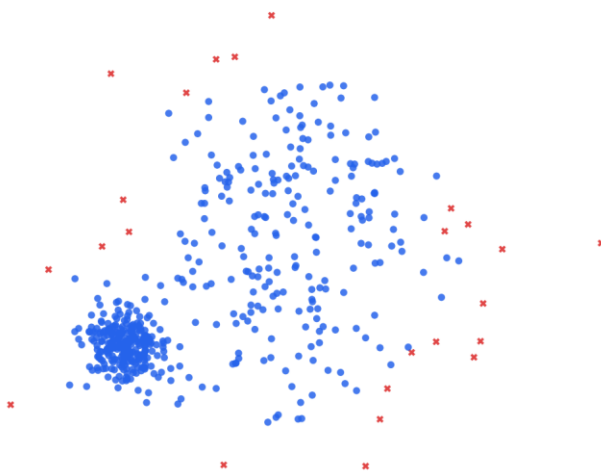
Where DBSCAN struggles

One global ϵ assumes every cluster has a similar density — when that fails, nothing works

$\epsilon = 0.14$: the sparse cluster dissolves into noise
1 cluster, 271 points called noise



$\epsilon = 0.45$: the two clusters merge into one
1 cluster, 24 points called noise



The structural weakness

A dense cluster beside a sparse one cannot be served by a single radius. Small ϵ dissolves the sparse cluster into noise; large ϵ merges both into one. There is no ϵ that is right for both.

Clusters of differing density

OPTICS — `sklearn.cluster.OPTICS`

builds a reachability ordering and extracts clusters at several density levels at once

...plus a hierarchy, fewer knobs

HDBSCAN — `sklearn.cluster.HDBSCAN` (≥ 1.3)

needs only `min_cluster_size`; usually the better default today

Only outliers matter, not clusters

LocalOutlierFactor · **IsolationForest**

purpose-built detectors with a score, not just a label

Very large n , or streaming

BIRCH, or **DBSCAN on a subsample**

then assign the rest with the nearest-core-point rule

10

What to take away

1

Density, not distance to a centre

Two parameters: ϵ and MinPts. Every point is core, border, or noise — and a cluster is a chain of overlapping dense neighbourhoods.

2

It answers two questions at once

How many regimes exist, and which readings belong to none of them. One algorithm, both the baseline study and the anomaly detector.

3

The recipe

Standardise $\rightarrow$ MinPts $\approx 2 \cdot d \rightarrow \epsilon$ from the k-distance knee $\rightarrow$ confirm the answer sits on a stable plateau.

4

Know the failure mode

One global ϵ cannot serve clusters of different density. Reach for OPTICS or HDBSCAN when it does.

Hands-on: DBSCAN_condition_monitoring.ipynb — the full pipeline, the ϵ sweep, the deployment rule, and seven exercises